Edge cases (search comes back empty) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `empty_search_practice.py`: an empty result list is the easy case to handle — a real vector search almost never returns nothing; it returns the *closest* chunks regardless of whether they're actually relevant, since distance never says "I don't know." This exercise handles both: a genuinely empty store, and a non-empty one whose best match is still too far (past a distance threshold) to be trusted. **If not:** the Build Task would answer from thin or empty context, confidently, which is exactly what its Constraints forbid.

Uses the same `practice/` setup as the Basic exercise (Doc08's files, `docs/` and `kb.py`). The only way to get a truly empty result is an empty store, so the code also builds one from an empty folder (Doc08's Build Task test did the same).

Solution

```
# empty_search_practice.py
from pathlib import Path
from typing import TypedDict

from langchain_openai import ChatOpenAI
from langgraph.graph import END, START, StateGraph

from ingest import build_vector_store
from kb import collection
from retriever import retrieve

model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
Path("empty_docs").mkdir(exist_ok=True)
empty_collection = build_vector_store("empty_docs", collection_name="empty")

class SearchState(TypedDict):
    task: str
    sources: list[dict]
    answer: str
    grounded: bool
```

```
def make_search_node(search_collection):
    def search_node(state: SearchState) -> dict:
        return {"sources": retrieve(search_collection, state["task"])}
    return search_node

def has_results(state: SearchState) -> str:
    if not state["sources"]:
        return "no_grounding"
    return "reason"

def reason_node(state: SearchState) -> dict:
    pieces: list[str] = []
    for source in state["sources"]:
        pieces.append(source["text"])
    context = "\n\n".join(pieces)
    prompt = f"Context:\n{context}\n\nQuestion: {state['task']}"
    return {"answer": model.invoke(prompt).content, "grounded": True}

def cant_answer_node(state: SearchState) -> dict:
    return {
        "answer": "I don't have grounding for this in my documents.",
        "grounded": False,
    }

def build_graph(search_collection, router=has_results):
    builder = StateGraph(SearchState)
    builder.add_node("search_node", make_search_node(search_collection))
    builder.add_node("reason_node", reason_node)
    builder.add_node("cant_answer_node", cant_answer_node)
    builder.add_edge(START, "search_node")
    builder.add_conditional_edges(
        "search_node", router,
        {"no_grounding": "cant_answer_node", "reason": "reason_node"},
    )
    builder.add_edge("reason_node", END)
    builder.add_edge("cant_answer_node", END)
    return builder.compile()

# Compare: a question answerable from the docs vs an empty store.
task = {"task": "How hot should the water be when brewing?"}
in_scope = build_graph(collection).invoke(task)
empty = build_graph(empty_collection).invoke(task)
print("coffee docs -> grounded:", in_scope["grounded"])
print("empty store -> grounded:", empty["grounded"])
```

Expected output:

```
coffee docs -> grounded: True
empty store -> grounded: False
```